



University
of Glasgow

W3-01: Hadoop Execution

COMPSCI4064 (H) and COMPSCI5088 (M)

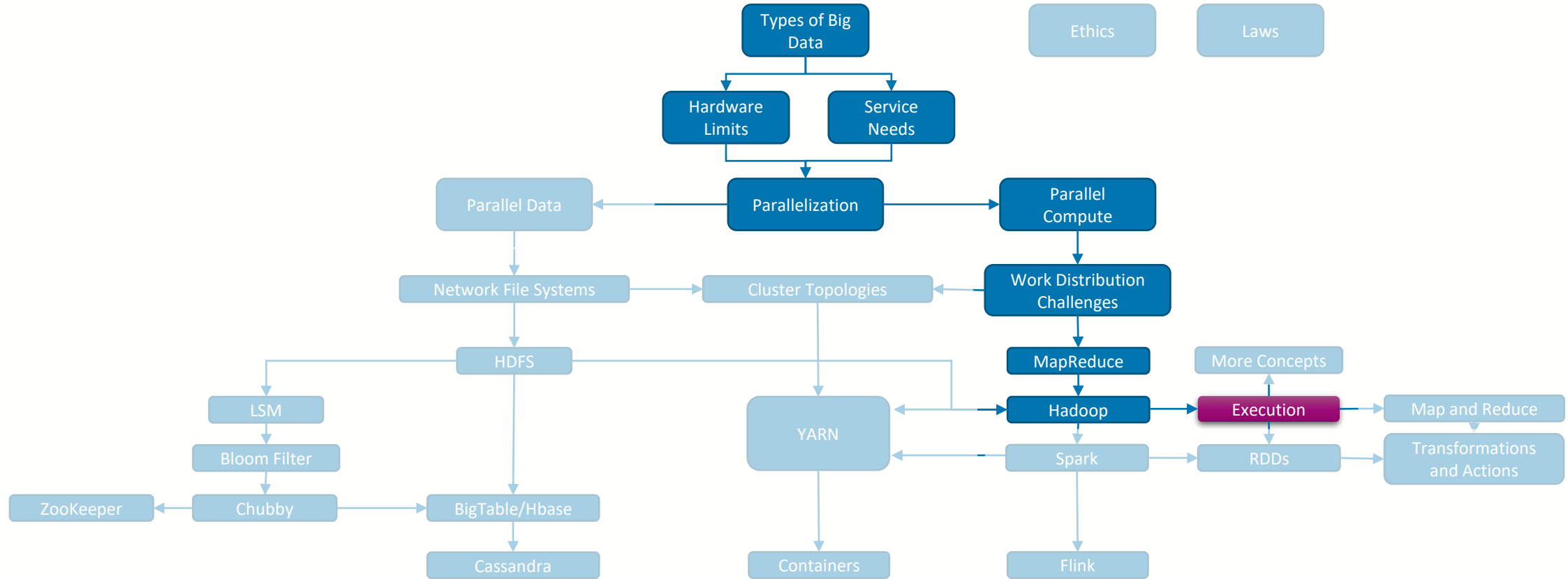
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



Where are we?



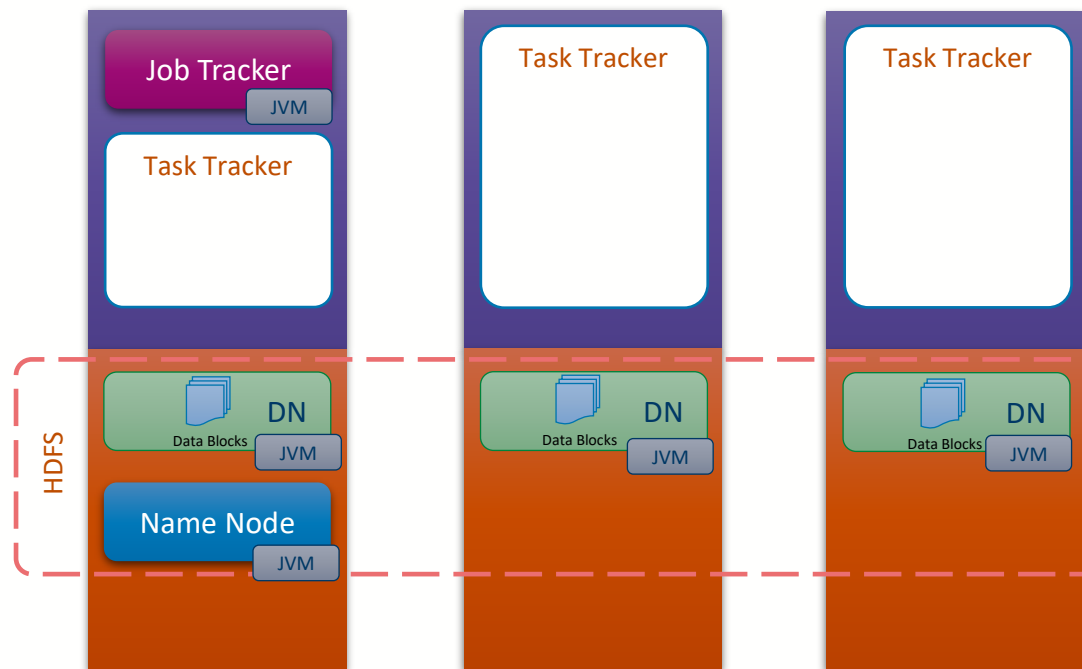


Job execution anatomy

1 Run Job



Client Machine



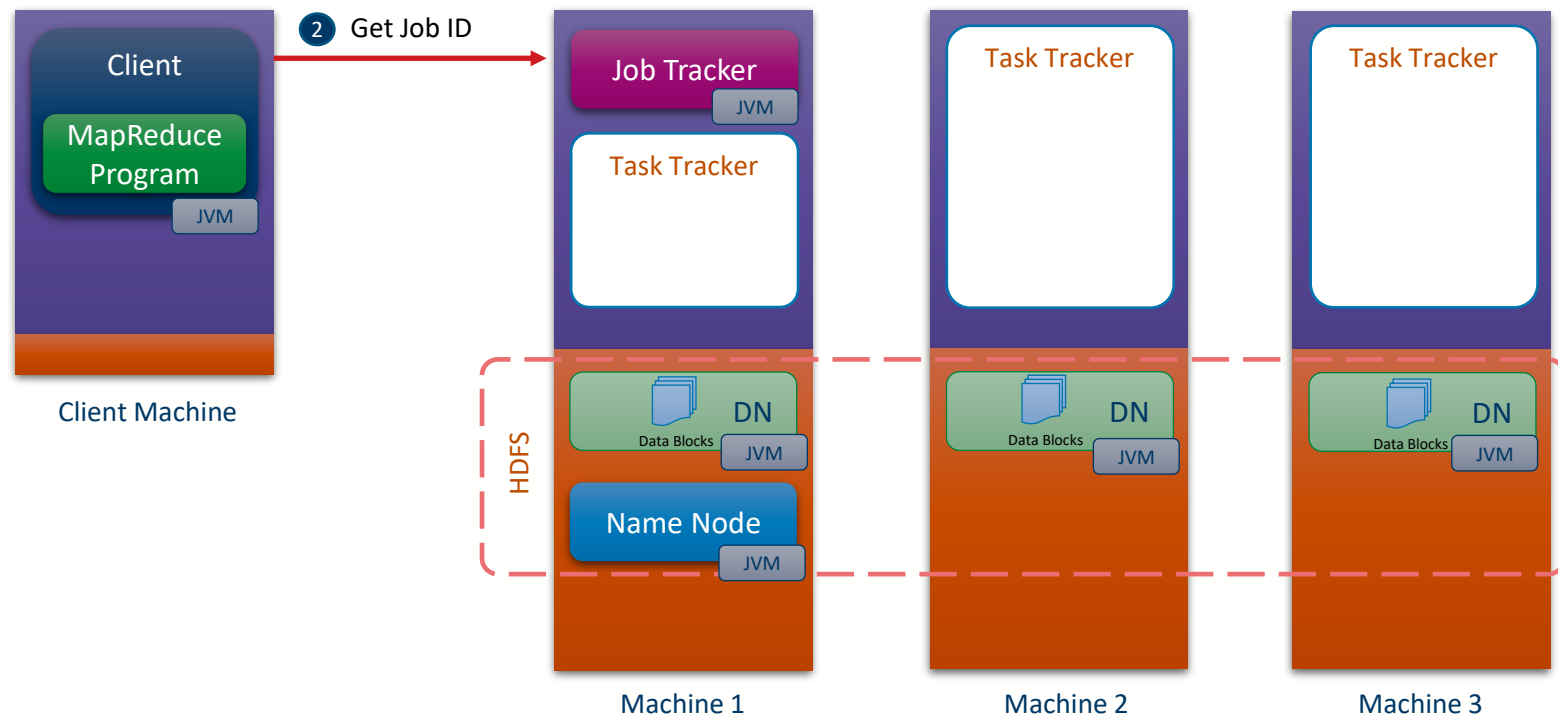
Machine 1

Machine 2

Machine 3

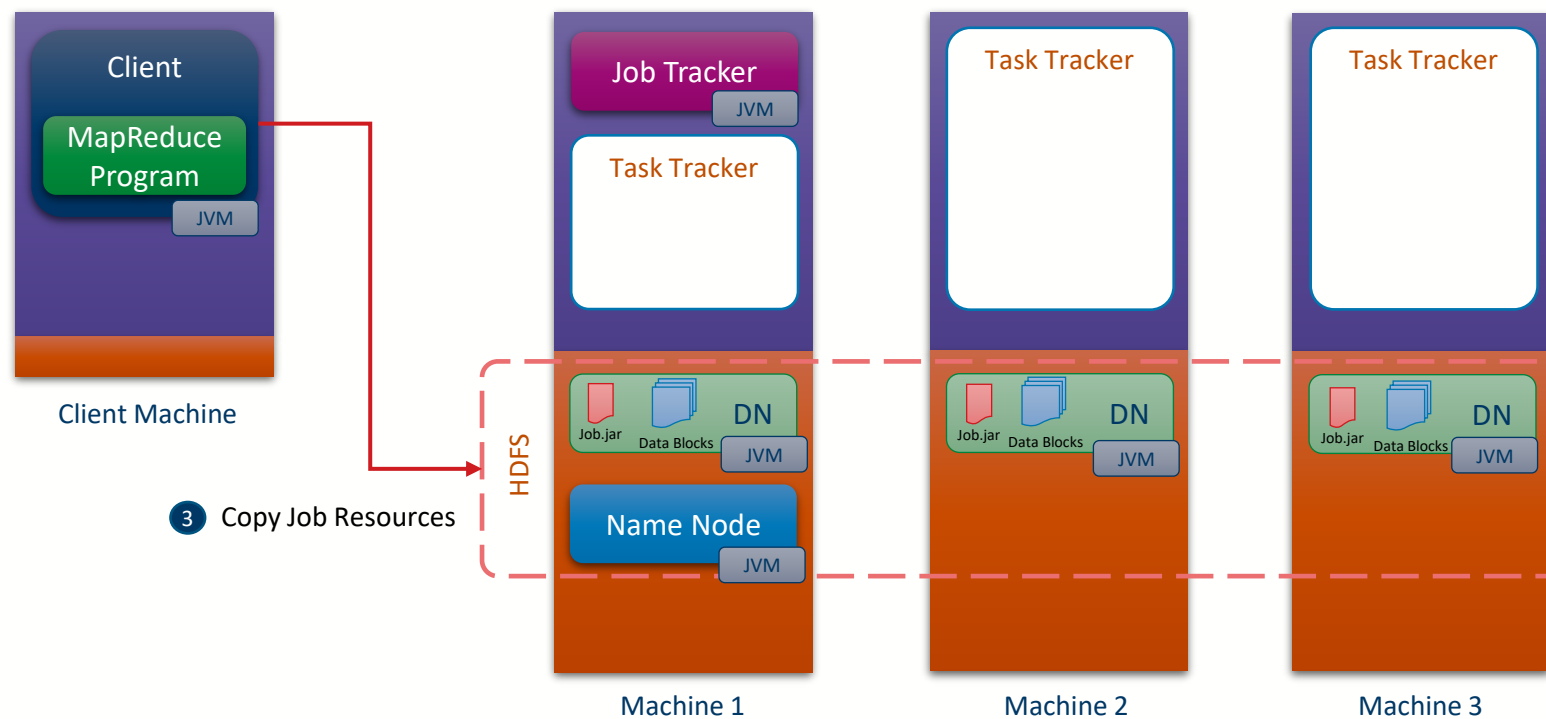


Job execution anatomy



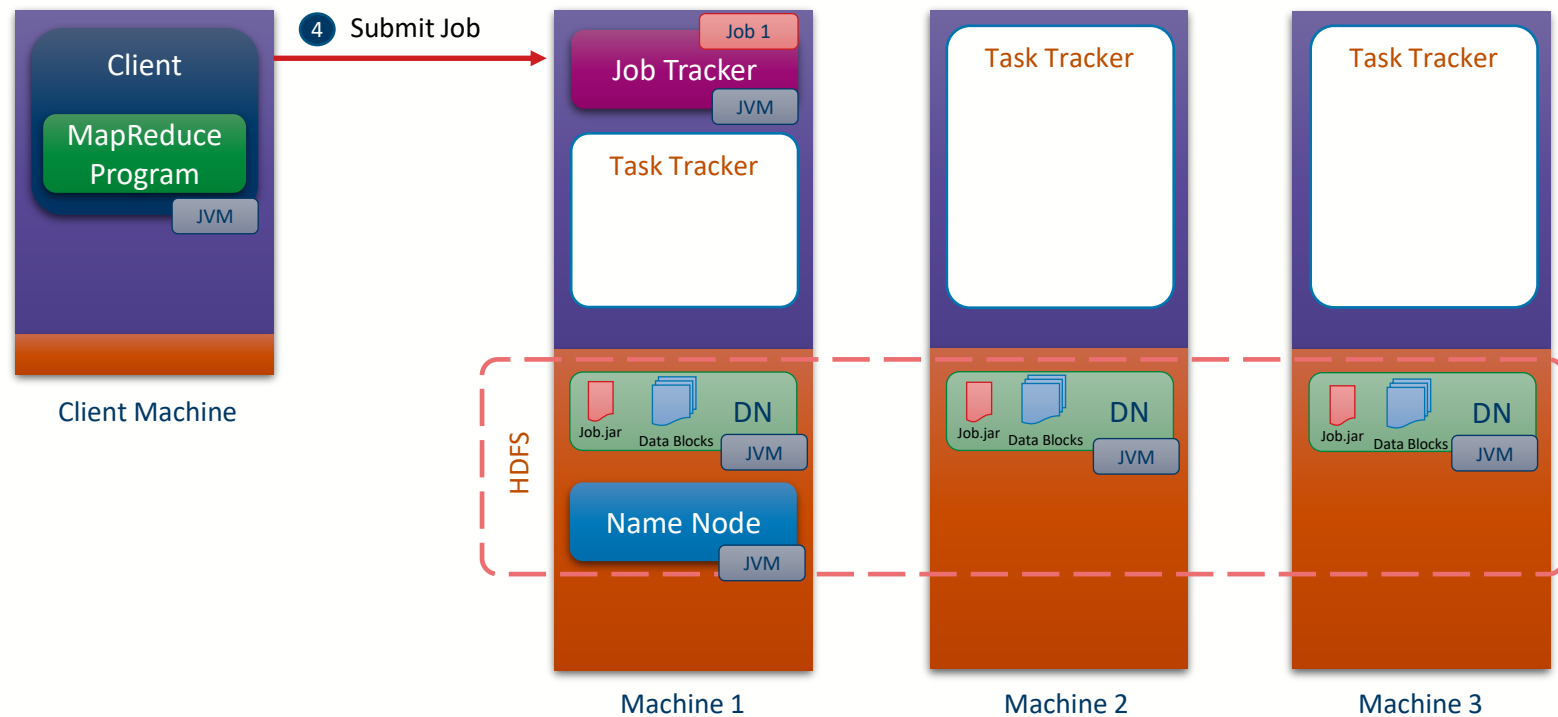


Job execution anatomy



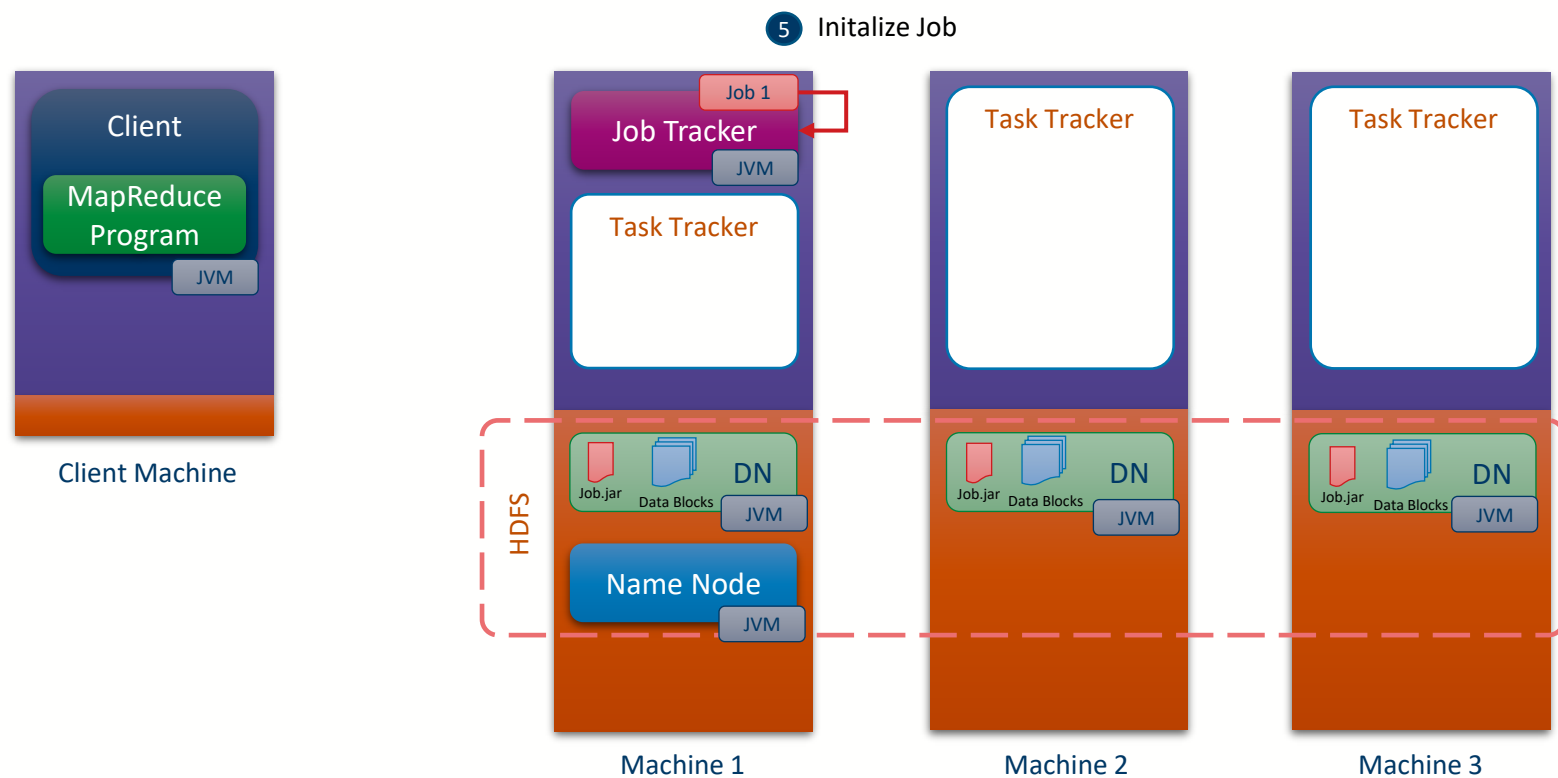


Job execution anatomy



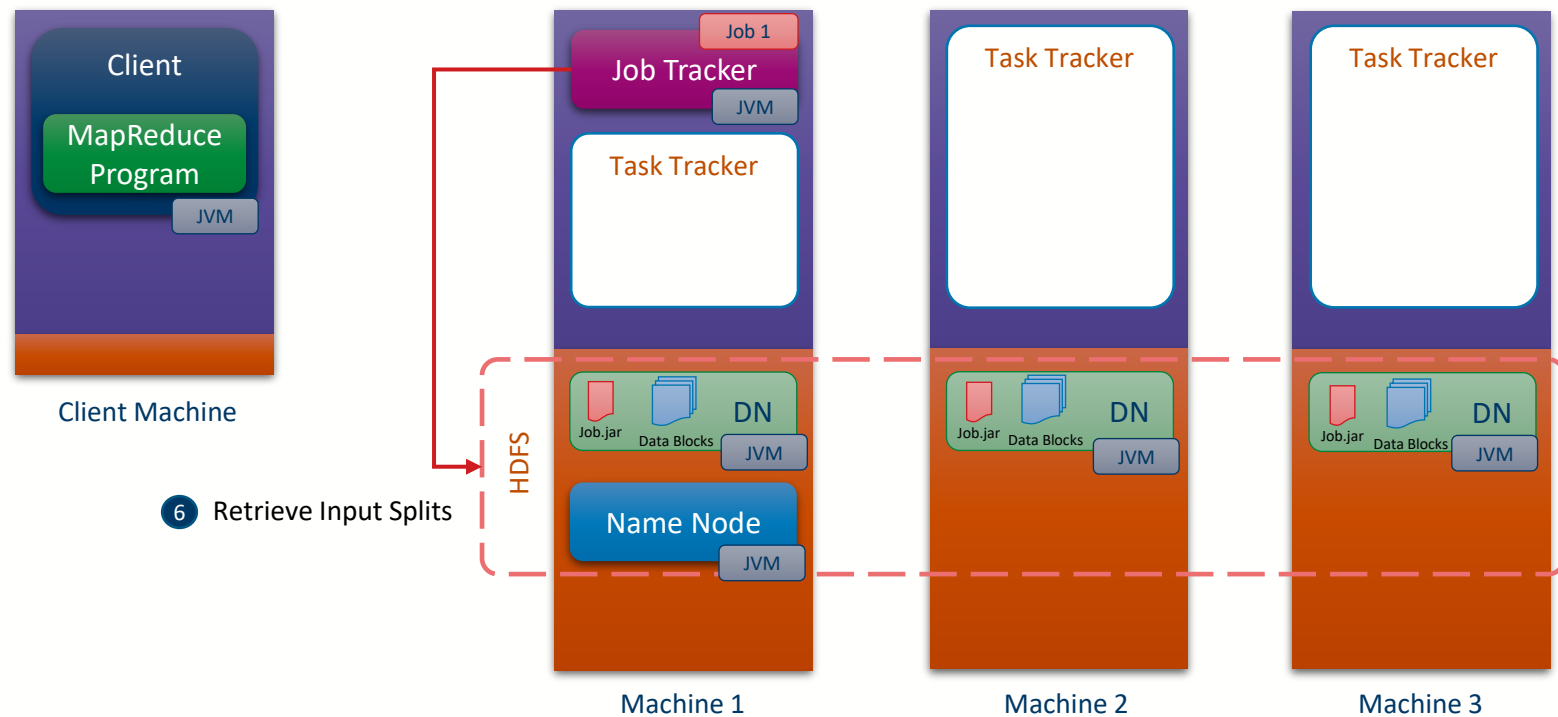


Job execution anatomy





Job execution anatomy



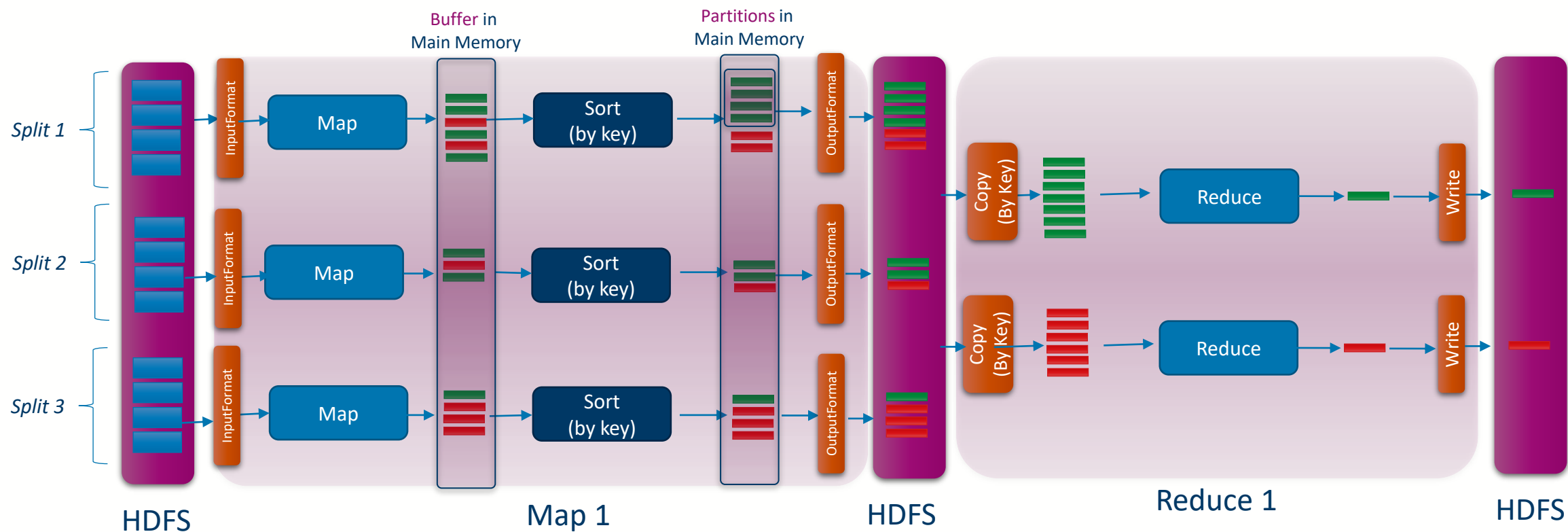


Input Splitting

- As discussed earlier, we want to evenly divide the work we are going to do across the tasks or pipelines that we have available
 - This involves **splitting the data**
- When we write data to the HDFS we usually **pre-compute and store information about how to split the data**
 - The default strategy is to **split each file into equally sized parts**
 - Splits should (by default) be **at most one HDFS block large** (this specifies a lower limit on the number of possible splits)
- Splits are **data agnostic** (they don't understand the format of the data)

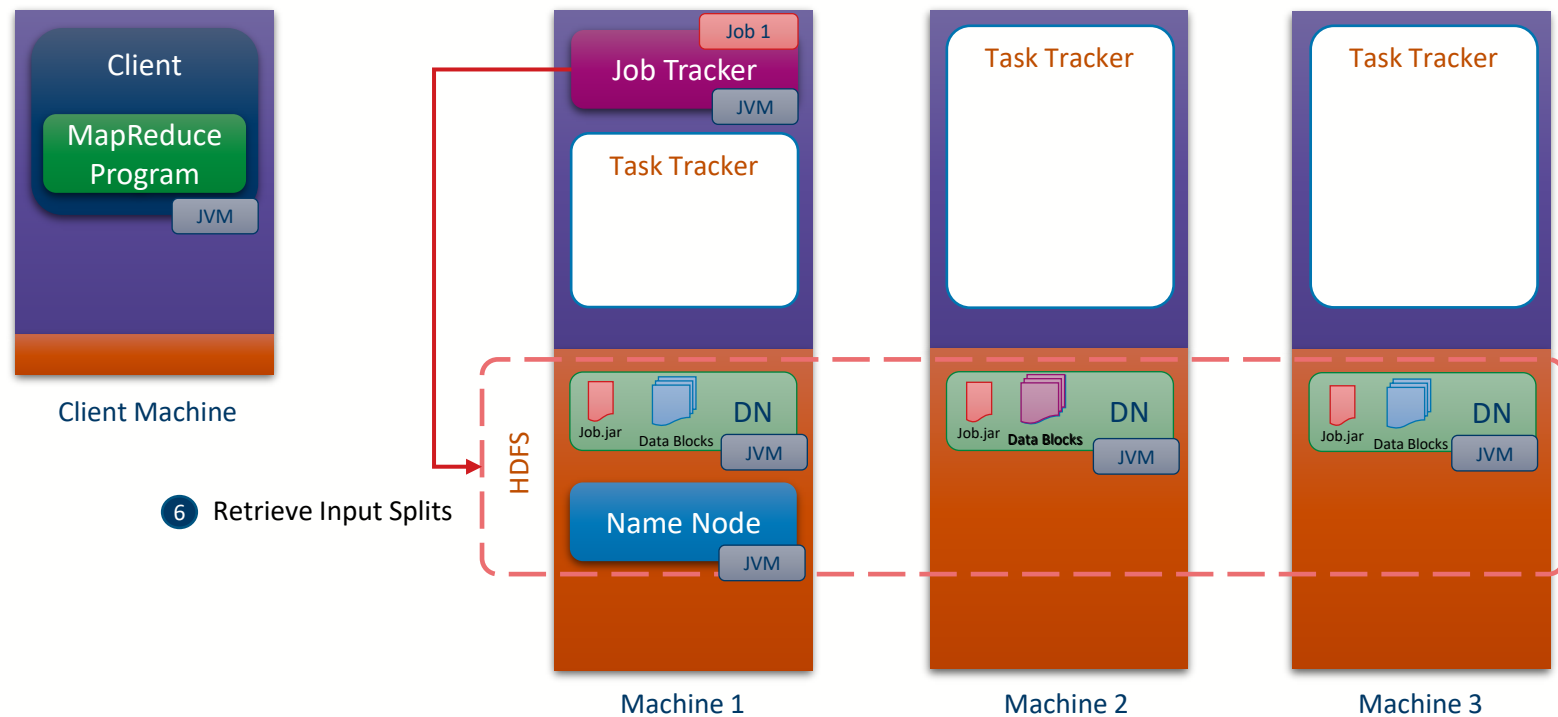


Input Splitting



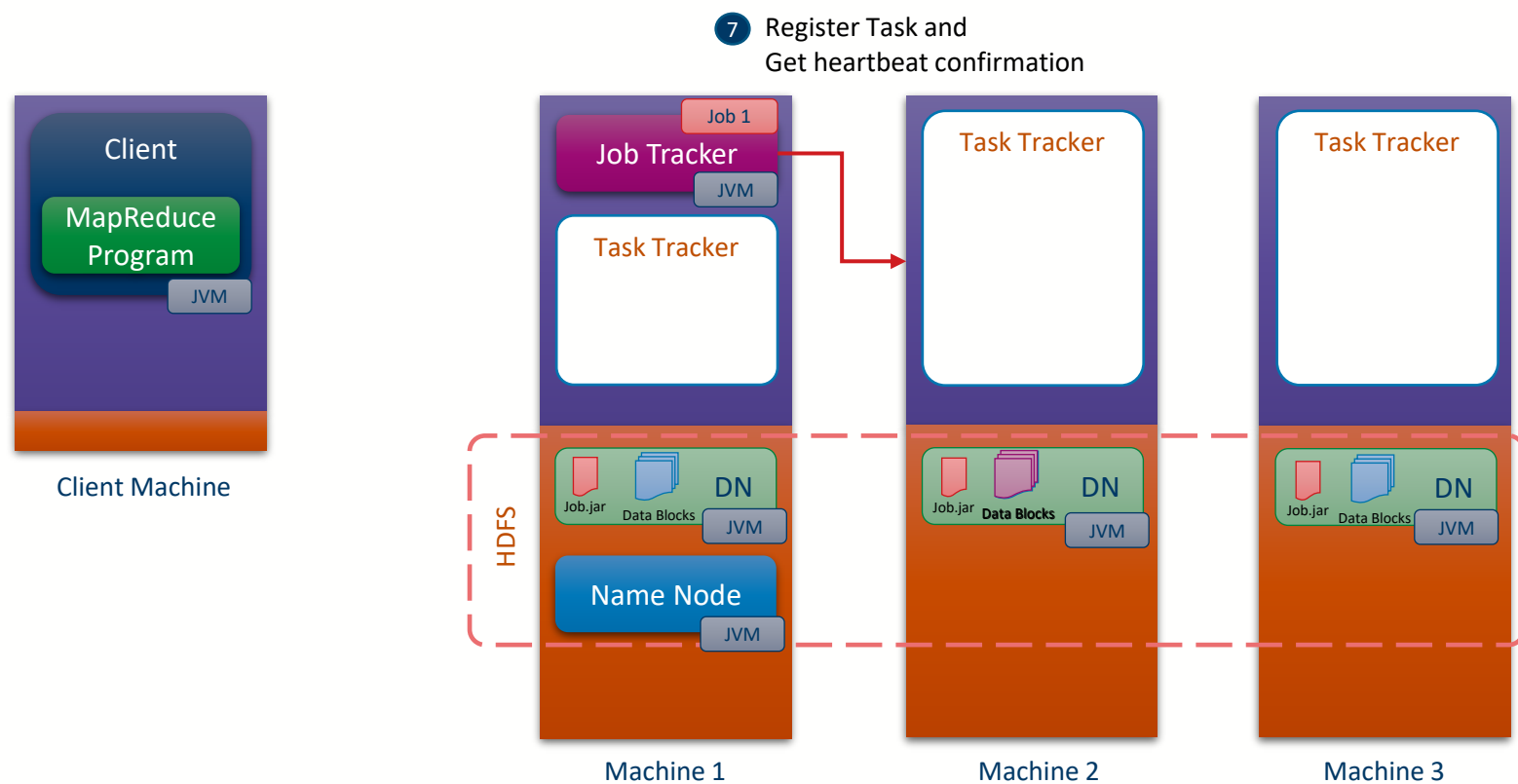


Job execution anatomy





Job execution anatomy



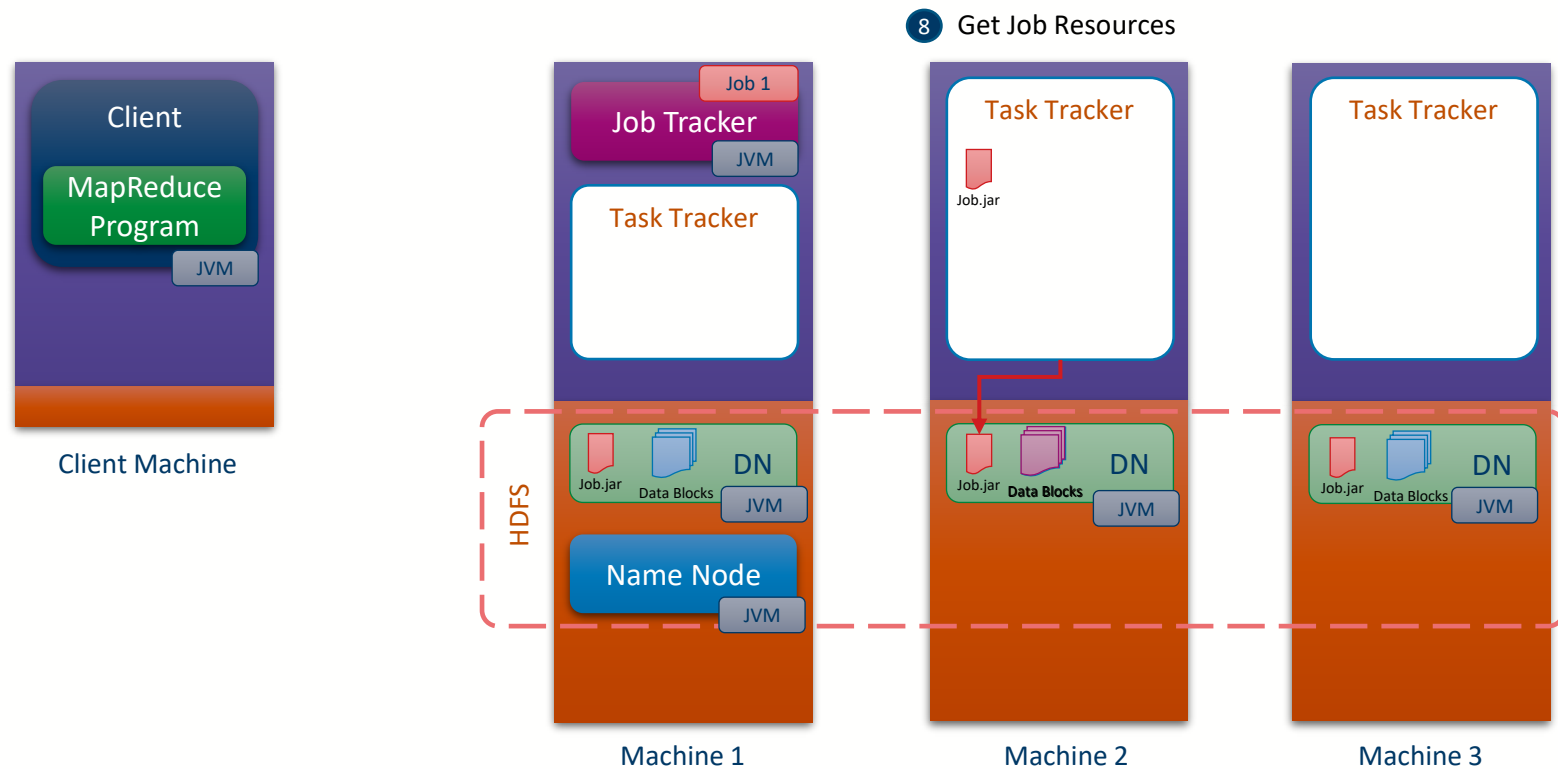


Task Assignment

- The JobTracker assigns each task to a TaskTracker
- How does the Job Tracker know which Task Trackers are **idle**?
 - Through Job Tracker-Task Tracker **heartbeats**
- How does the Job Tracker decide which task to run where?
 - That's a job for the scheduler
 - Variants: **FIFO** (first in first out) / **Fair** / **Capacity**
 - Optimises for: Data-local > rack-local > remote
- In some configurations the Job Tracker may also use **speculative execution**
 - (a.k.a., backup task execution)

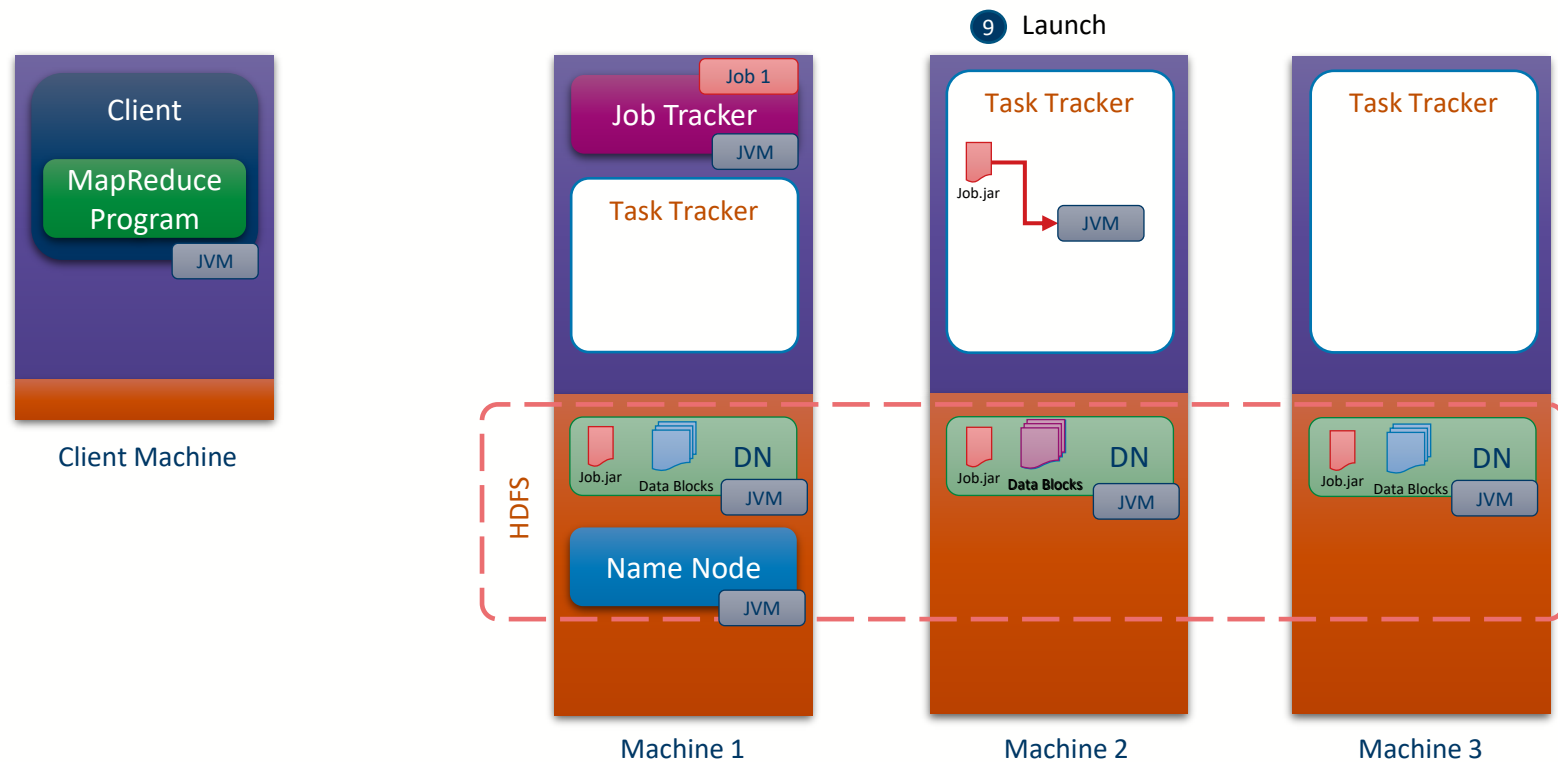


Job execution anatomy



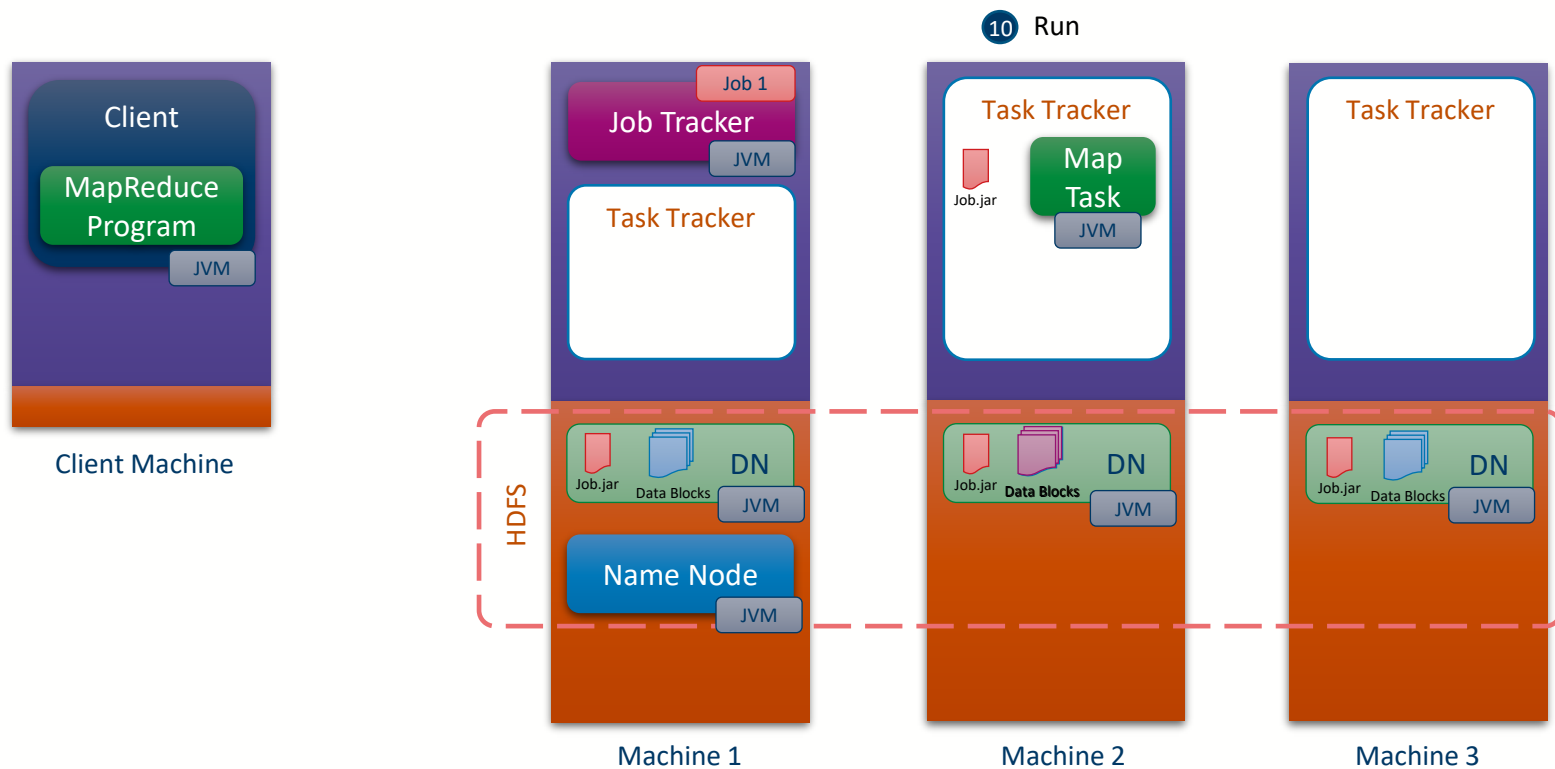


Job execution anatomy



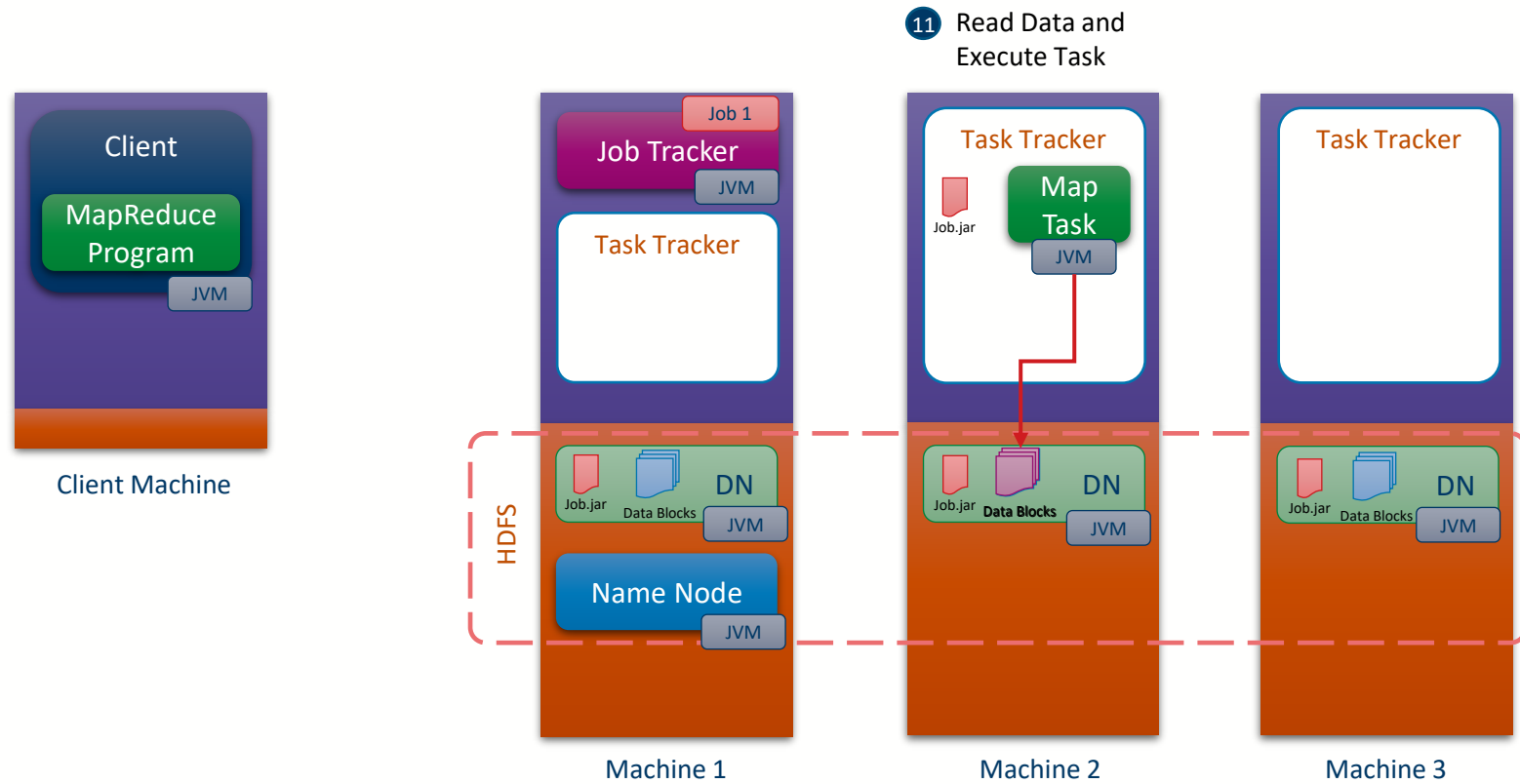


Job execution anatomy





Job execution anatomy



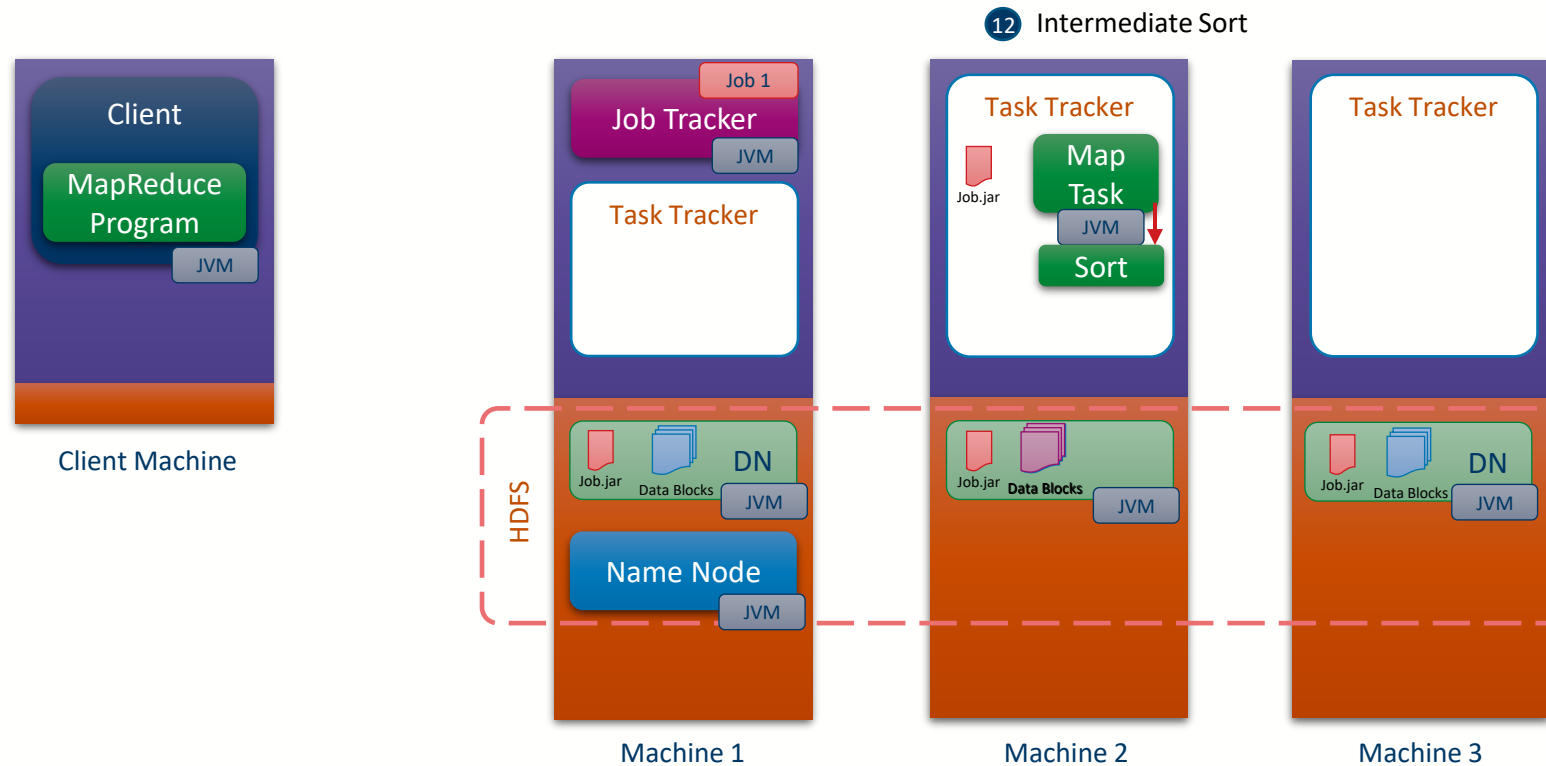


InputFormat

- **InputFormat** is the class that is responsible for converting from an HDFS block or blocks to usable data that can be passed to a Map or Reduce function
 - `abstract class org.apache.hadoop.mapreduce.InputFormat`
- We usually will use built-in InputFormats for common types of data
 - **File**InputFormat: Read data from files on HDFS; "utility" base class
 - **Text**InputFormat: Treat input as text files; line-oriented operation
 - **KeyValueText**InputFormat: Treat input as files storing key-value pairs, one per line; key-values separated by a special character (default: tab)
 - **Sequence**FileInputFormat: Treat input as binary files storing key-value pairs (i.e., `org.apache.hadoop.io.SequenceFile`'s)
 - `SequenceFileAsTextInputFormat`: Converts keys/values to Strings by means of their `toString()` method
 - `SequenceFileAsBinaryInputFormat`: Reads keys/values in their raw format
 - **CombineFile**InputFormat: Process multiple small files as a single large file
 - `CombineTextInputFormat`: Combine multiple text files
 - `CombineSequenceFileInputFormat`: Combine multiple `SequenceFile`'s
 - **NLine**InputFormat: Treat input as text files, create a split every N lines
 - **DB**InputFormat: Read data from a SQL table (over JDBC)
 - **Composite**InputFormat: "Join" multiple data sources

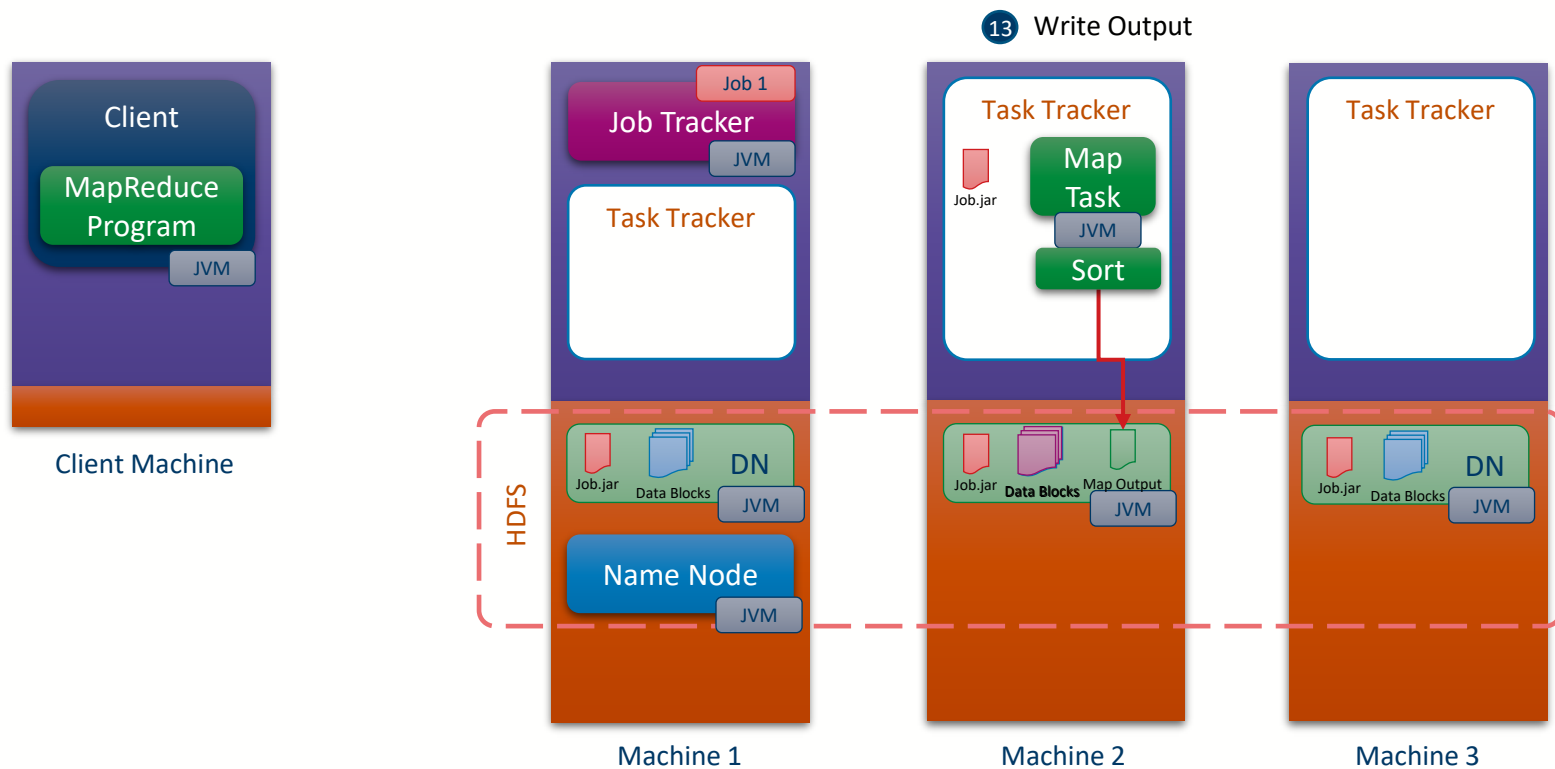


Job execution anatomy



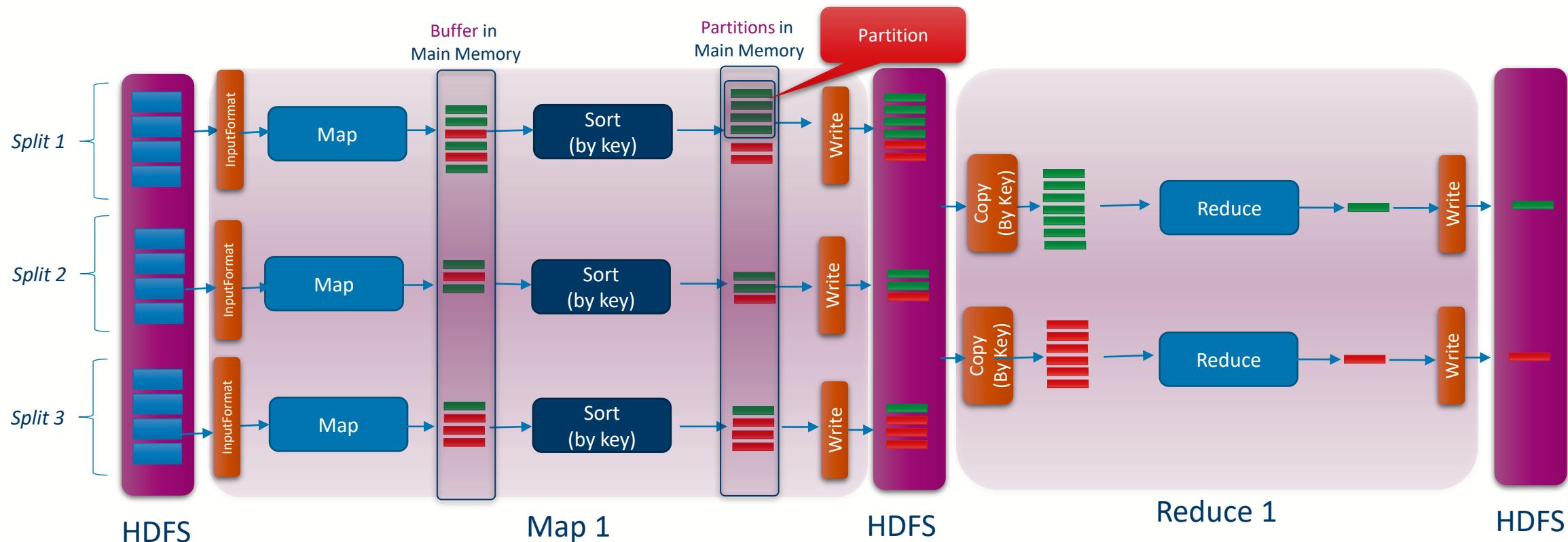


Job execution anatomy



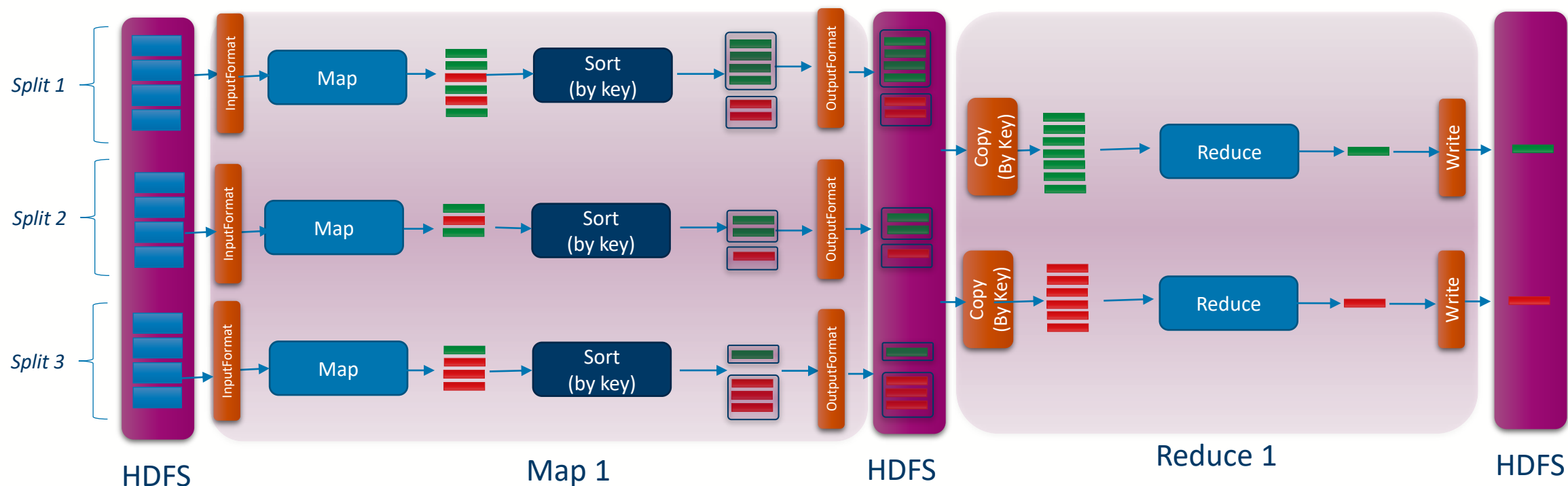
Intermediate output and shuffling

- **Sorting** and **Partitioning** of data is performed in Main Memory to save time



Intermediate output and shuffling

- To write our output partition(s) to file we use an **OutputFormat**



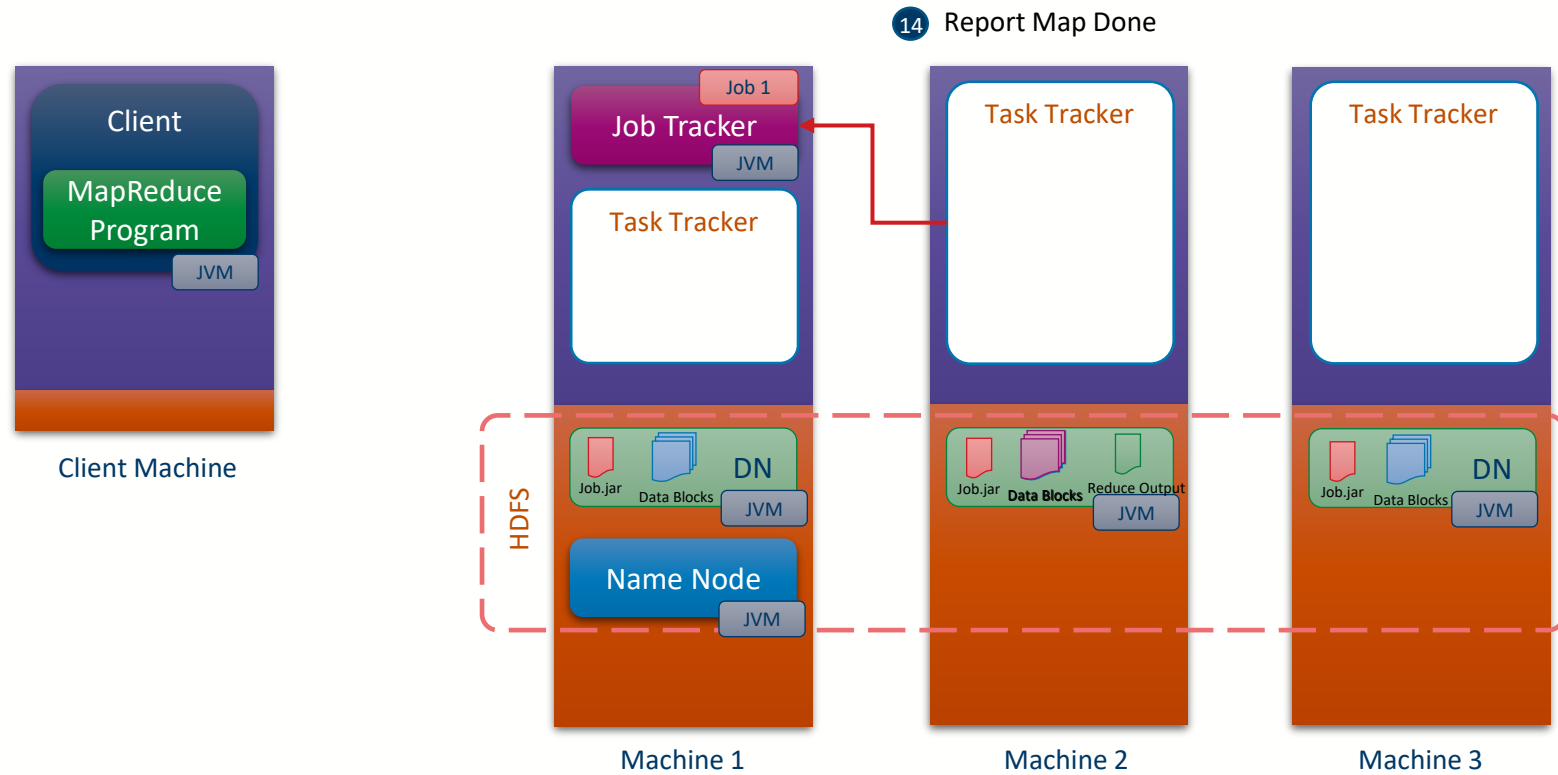


OutputFormat

- OutputFormat classes are responsible for writing partitions of data to persistent storage (usually to the HDFS)
 - `abstract class org.apache.hadoop.mapreduce.OutputFormat`
- Like InputFormat, there are a range of pre-provided writers available
 - **File**OutputFormat: Write data to files on HDFS
 - **Text**OutputFormat: Write plain text files
 - **MapFile**OutputFormat: Write indexed key-value files
 - **SequenceFile**OutputFormat: Write output in `org.apache.hadoop.io.SequenceFile` format
 - `SequenceFileAsBinaryOutputFormat`: Write output in raw format
 - **Null**OutputFormat: Send all output to oblivion (a.k.a. `/dev/null`)
 - **Lazy**OutputFormat: Writes data out in a lazy manner
 - **MultipleOutputs**: Writes data to multiple files, possibly using a different OutputFormat for each

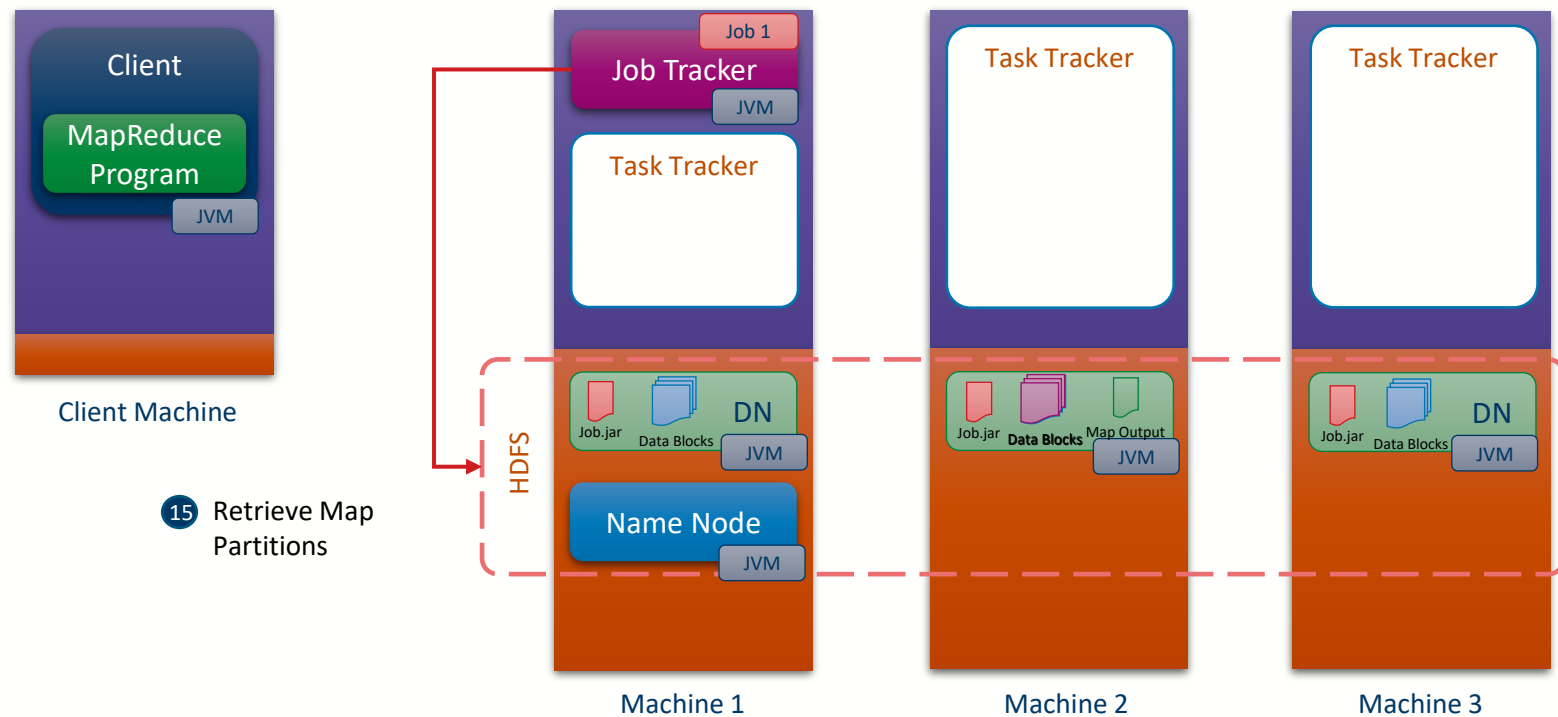


Job execution anatomy



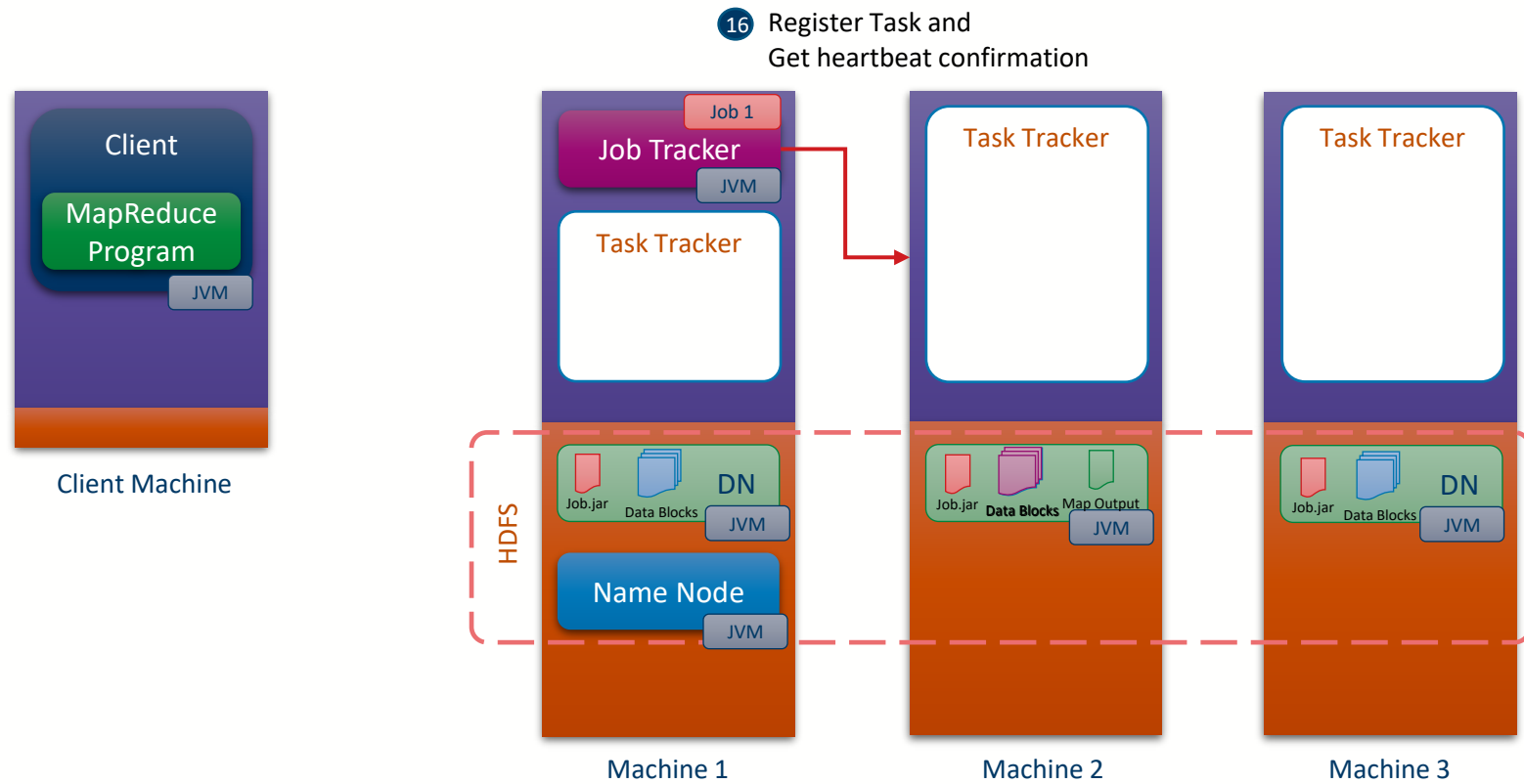


Job execution anatomy



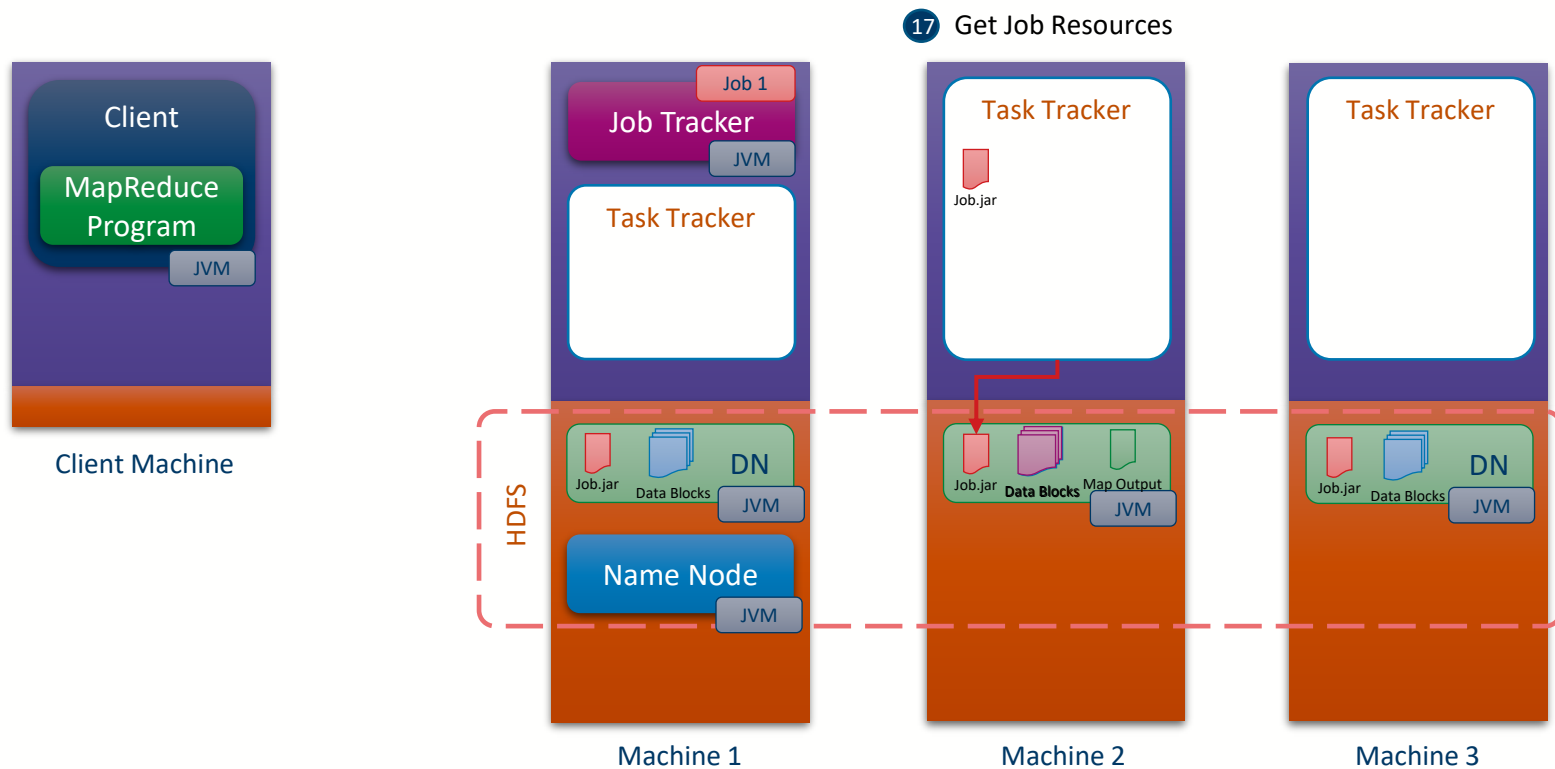


Job execution anatomy



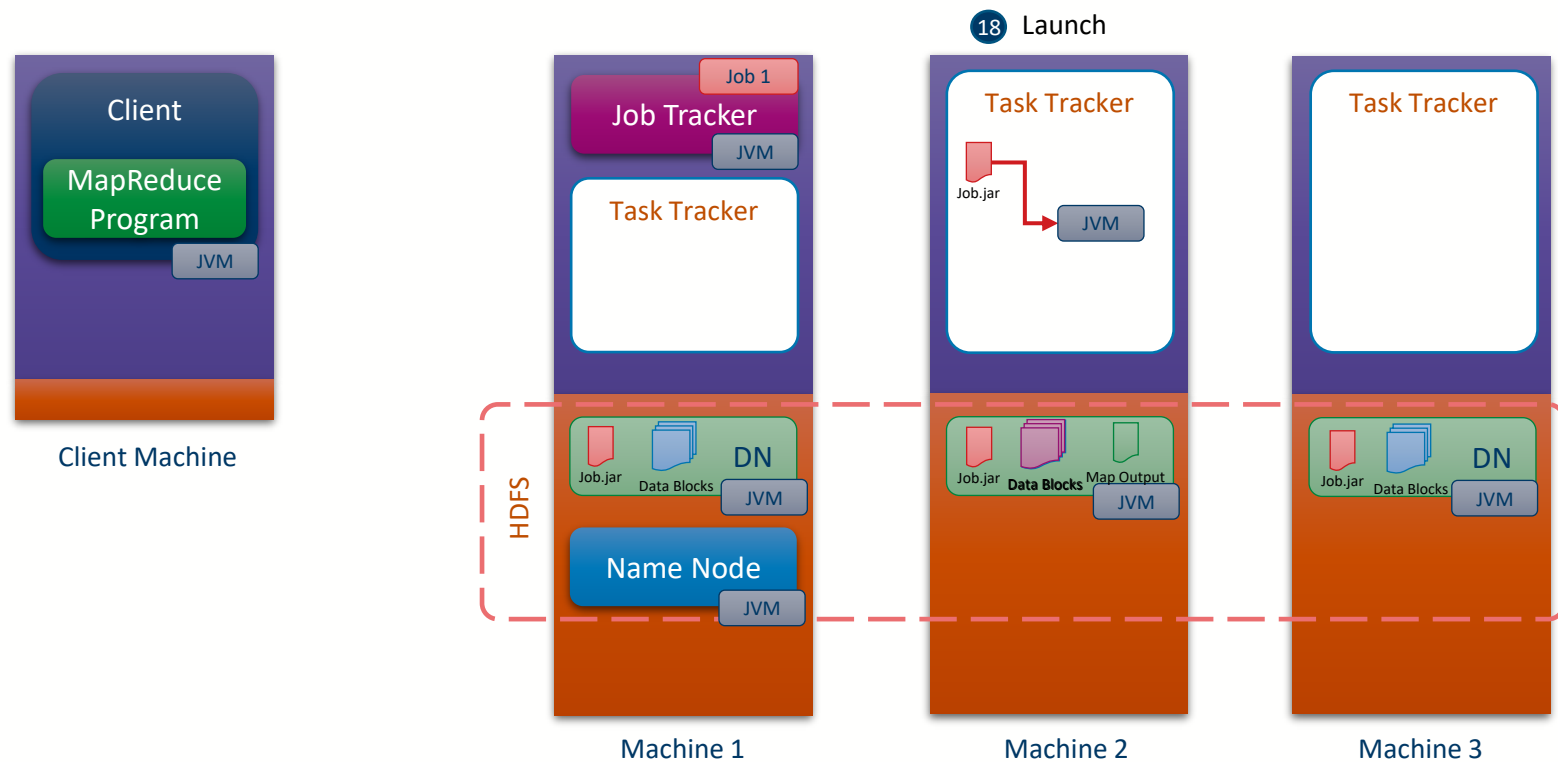


Job execution anatomy



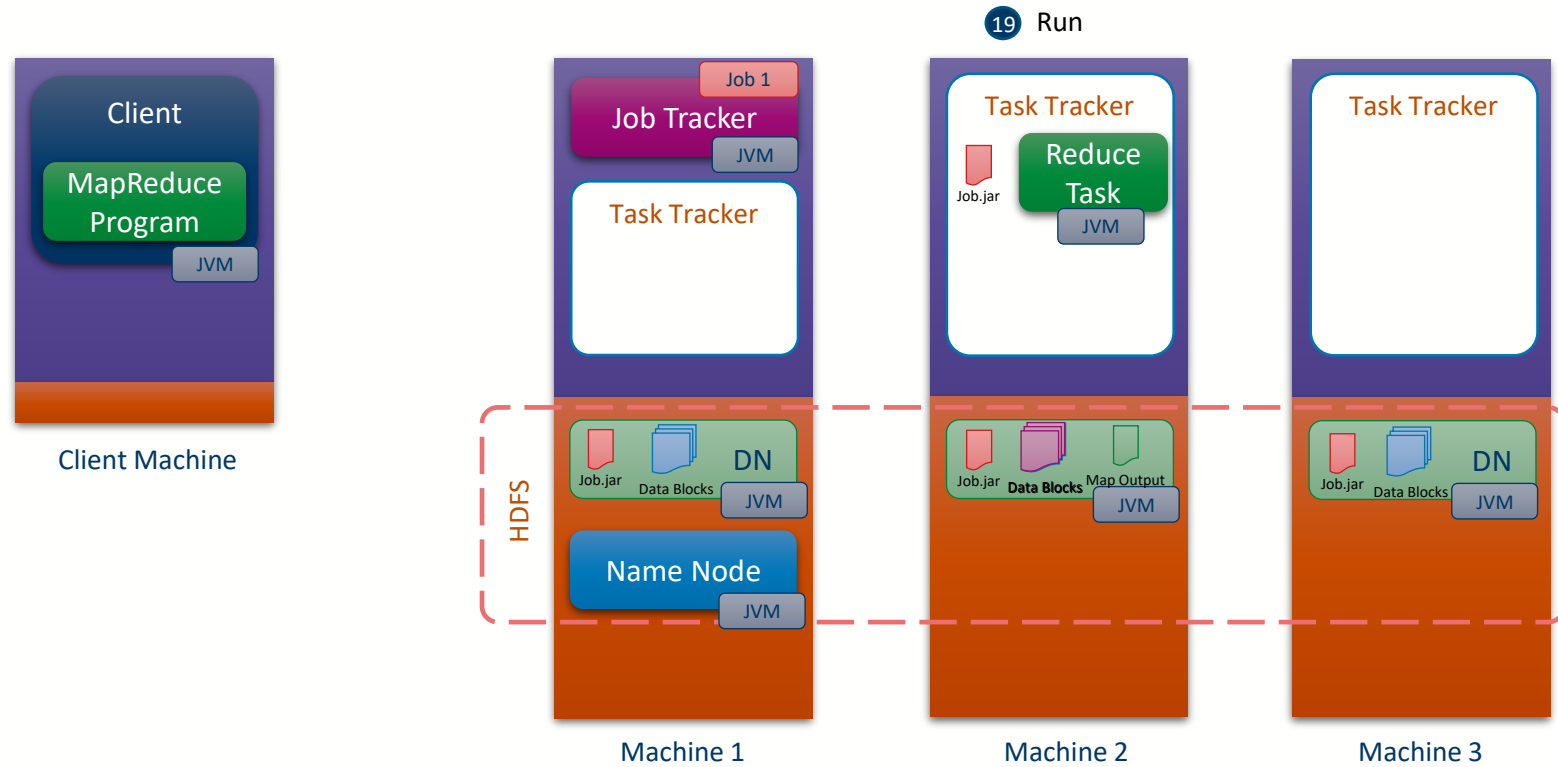


Job execution anatomy



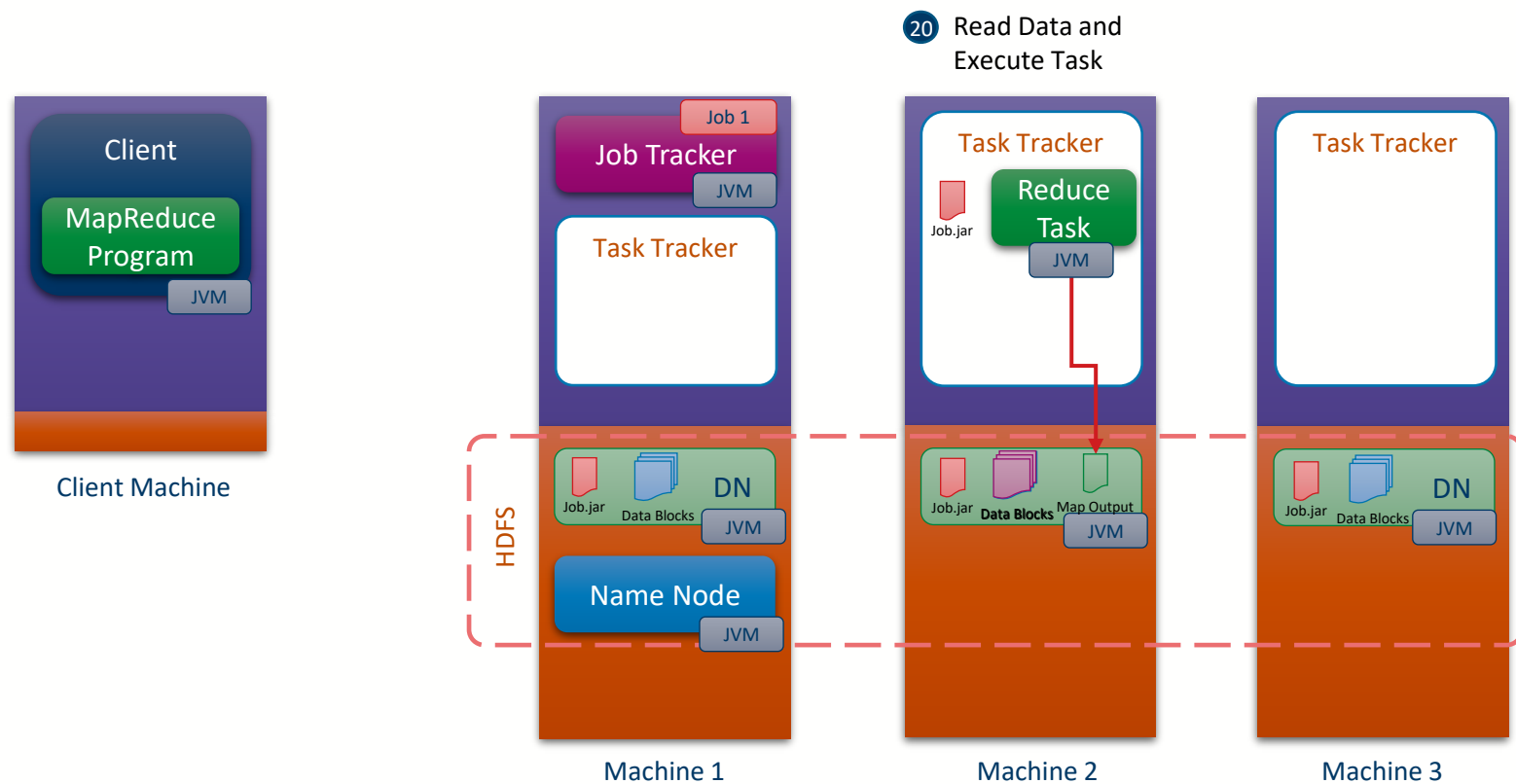


Job execution anatomy



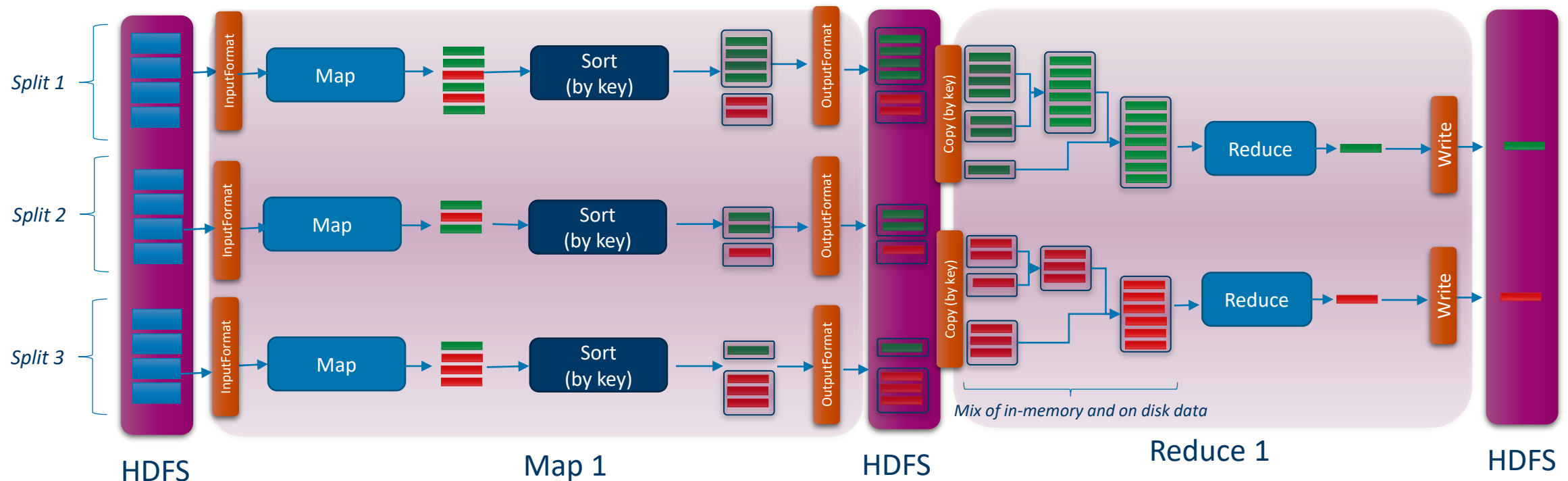


Job execution anatomy



Intermediate output and shuffling

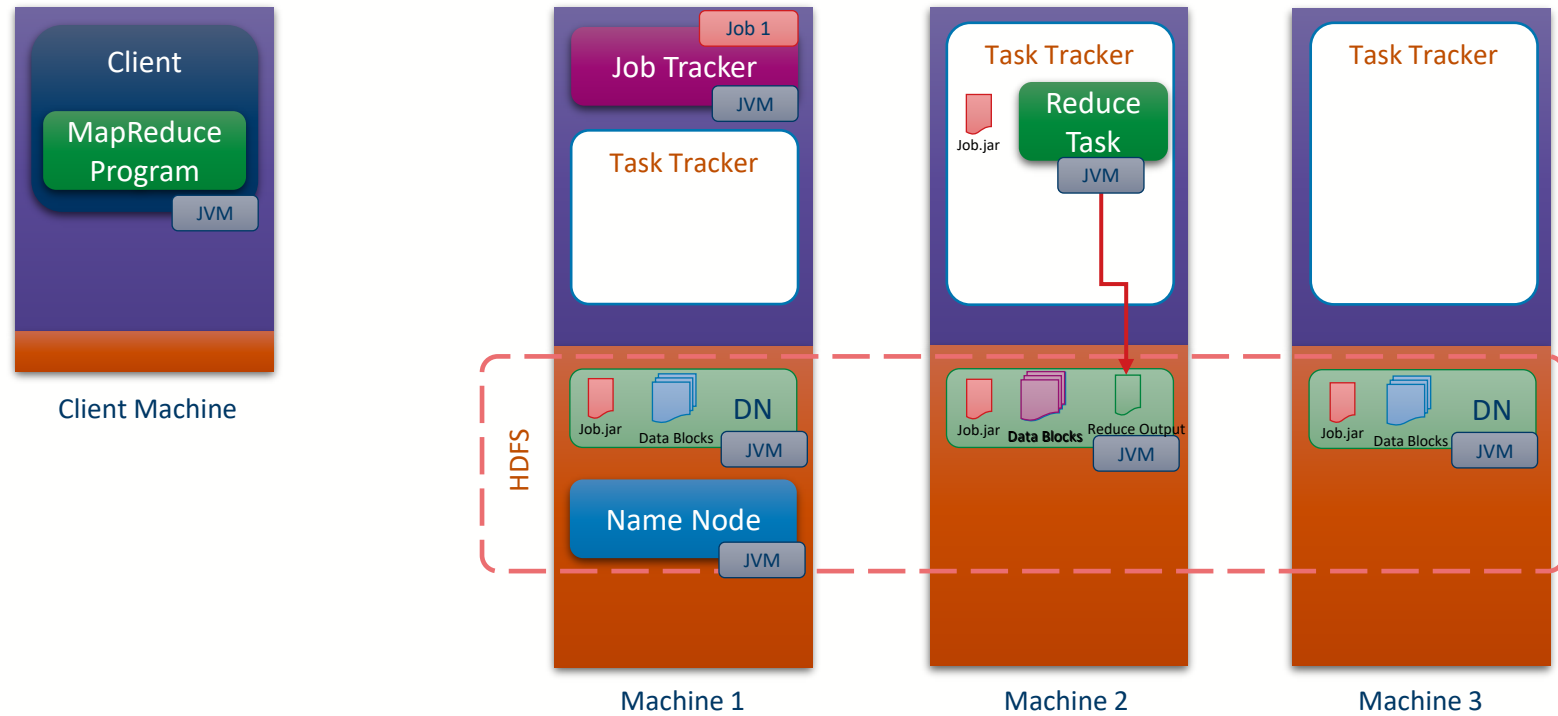
- Data is loaded into a reducer via a **recursive merge of partitions**





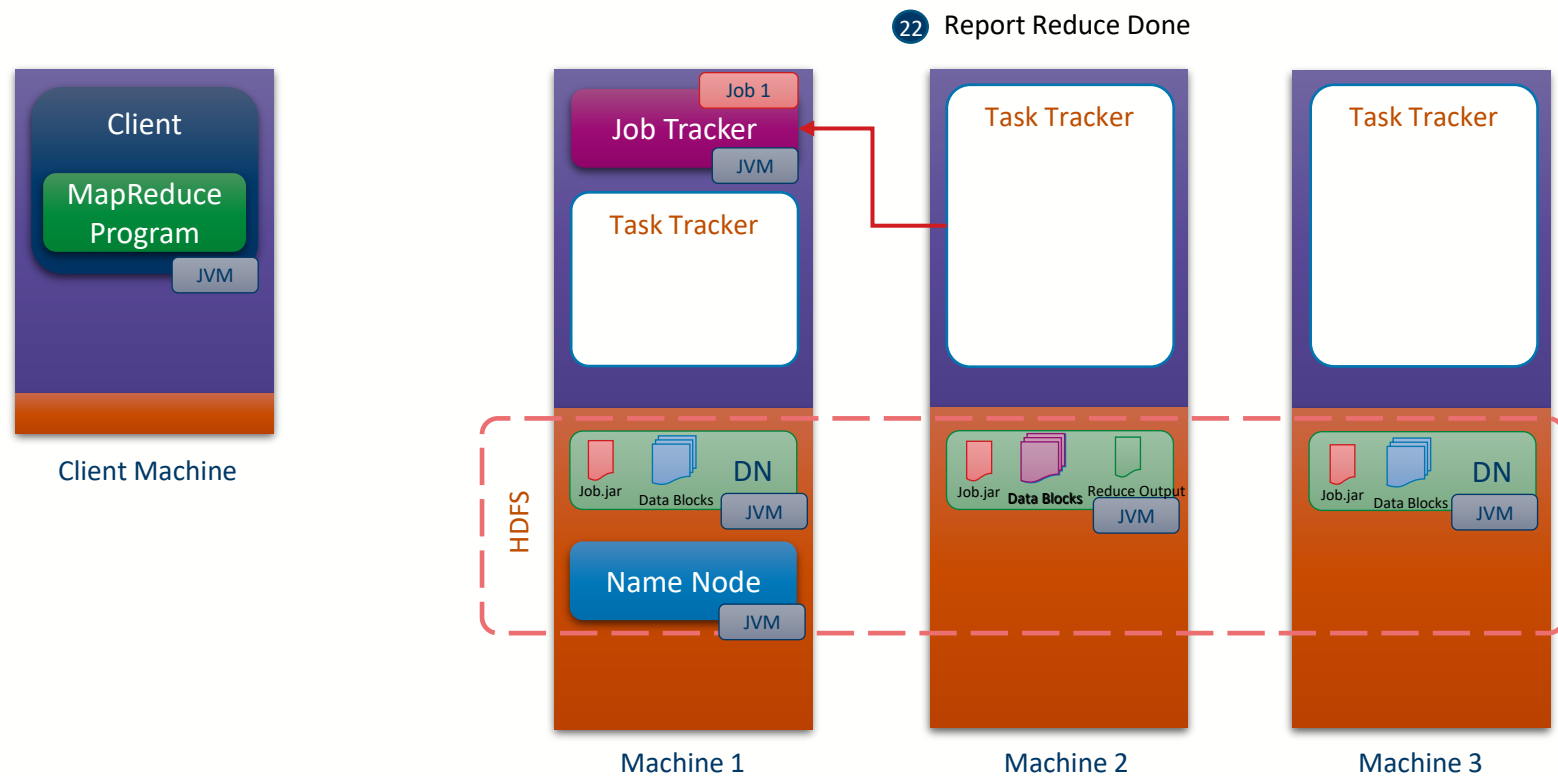
Job execution anatomy

21 Write Reduce Data



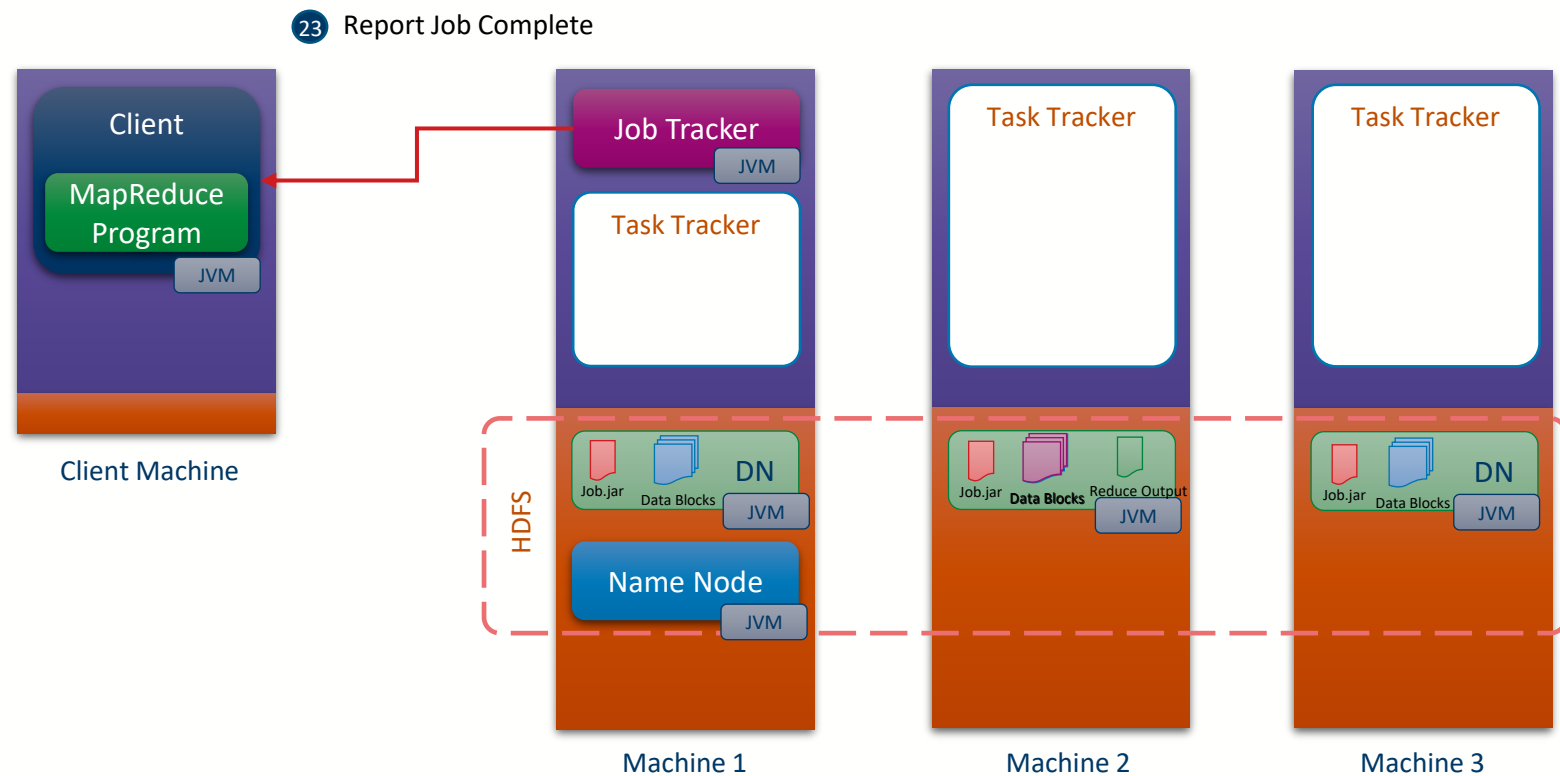


Job execution anatomy





Job execution anatomy





University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow